DEUTCHEZZ HOTEL ISPAHAN

BACKEND DOMAIN MODEL, BUSINESS RULES & API CONTRACT

SOURCE OF TRUTH — VERSION 1.0

Location: Ahmadu Bello Way, Jos, Plateau State, Nigeria

Purpose: Canonical backend design reference to be used before and during implementation.

Item	Agreed baseline
Backend	ASP.NET Core / C#
Database	PostgreSQL
ORM	Entity Framework Core
Authentication	ASP.NET Core Identity / secure JWT-based API authentication
Frontend consumer	React + TypeScript + Vite + Tailwind + shadcn/ui
Current payment model	Pay at Hotel / receptionist-recorded payments
Online gateway	Deferred until provider account and requirements are available
Inventory	Dynamic; initial 20 rooms are sample data, not a maximum

Governance rule: implementation should follow this document. If a business or architectural requirement changes, update the source of truth first and version the change before altering the affected implementation.

1. SYSTEM SCOPE

- Public hotel website and room discovery.
- Date-based room availability search.
- Anonymous and registered-user bookings.
- Multi-room bookings in one reservation.
- Customer booking dashboard and anonymous booking lookup.
- Receptionist reservation, room assignment, check-in, check-out, guest and payment operations.
- Administrator inventory, room type, staff, guest, payment, reporting, notification, settings and audit operations.
- Backend-controlled pricing, totals, authorization, state transitions and availability.

2. ARCHITECTURAL PRINCIPLES

- The backend is authoritative; frontend totals and status values are never trusted.
- Anonymous and registered bookings use the same Booking domain.
- A reservation describes requested room types and quantities; physical room assignment is a separate operational concern.
- Historical reservation rates are stored so later price changes do not alter existing bookings.
- Booking status and payment status are separate.
- Inventory-changing operations require appropriate transactions and concurrency protection.
- Important operational and administrative actions are auditable.

```
React Frontend → ASP.NET Core API → Application/Domain → Infrastructure/EF Core → PostgreSQL
```

3. BASE CLASSES

Base class	Members	Use
BaseEntity	Id	Common primary key.
AuditableEntity	CreatedAt, UpdatedAt	Creation/modification timestamps.
SoftDeletableEntity (selective)	IsDeleted, DeletedAt	Only for entities where non-destructive deletion is required.

Bookings and payments should normally be retained for history/audit rather than physically deleted.

4. ENUMS

Enum	Values	Purpose
UserRole	Customer, Receptionist, Administrator	Authorization.
RoomStatus	Available, Occupied, Maintenance, Inactive	Physical room operational state.
BookingStatus	Pending, Confirmed, CheckedIn, CheckedOut, Cancelled	Reservation lifecycle.
PaymentStatus	Pending, PartiallyPaid, Paid, Refunded, Failed	Payment state.
PaymentMethod	Cash, POS, BankTransfer, Online	Payment channel.
NotificationType	BookingConfirmed, PaymentReceived, BookingCancelled, CheckInCompleted, CheckOutCompleted	Notification classification.

5. DOMAIN ENTITIES

5.x User

Fields: Id, FirstName, LastName, Email, PhoneNumber, identity-managed credentials, Role, IsActive, CreatedAt, UpdatedAt, LastLoginAt

Purpose: Authenticated application account.

5.x Guest

Fields: Id, UserId (nullable), FirstName, LastName, Email, PhoneNumber, CreatedAt, UpdatedAt

Purpose: Hotel guest. UserId is nullable for anonymous guests.

5.x RoomType

Fields: Id, Name, Description, BasePrice, MaximumOccupancy, BedType, IsActive, CreatedAt, UpdatedAt

Purpose: Configurable room category and current base pricing.

5.x Room

Fields: Id, RoomNumber, RoomTypeId, Floor, Status, IsActive, CreatedAt, UpdatedAt

Purpose: Physical hotel room; administrators can add rooms dynamically.

5.x Booking

Fields: Id, BookingReference, GuestId, CheckInDate, CheckOutDate, Adults, Children, Status, PaymentStatus, SpecialRequests, TotalAmount, CreatedAt, UpdatedAt

Purpose: Reservation aggregate root.

5.x BookingRoomType

Fields: Id, BookingId, RoomTypeId, Quantity, RatePerNight, NumberOfNights, Subtotal

Purpose: Reservation line describing room type and quantity; preserves agreed rate.

5.x RoomAssignment

Fields: Id, BookingId, BookingRoomTypeId, RoomId, AssignedAt, AssignedByUserId

Purpose: Maps reserved quantities to actual physical rooms.

5.x Payment

Fields: Id, BookingId, Amount, PaymentMethod, Status, Reference, PaidAt, RecordedByUserId, Notes, CreatedAt

Purpose: One payment event; multiple payments may belong to one booking.

5.x Notification

Fields: Id, UserId, Type, Title, Message, IsRead, CreatedAt

Purpose: In-app notification for authenticated users.

5.x ActivityLog

Fields: Id, UserId, Action, EntityType, EntityId, Description, CreatedAt

Purpose: Audit trail for important actions.

6. ENTITY RELATIONSHIPS

```
Guest 1 ■■■ * Booking
Booking 1 ■■■ * BookingRoomType
RoomType 1 ■■■ * BookingRoomType
Booking 1 ■■■ * RoomAssignment
BookingRoomType 1 ■■■ * RoomAssignment
Room 1 ■■■ * RoomAssignment (across different reservations/dates)
Booking 1 ■■■ * Payment
User 1 ■■■ * Notification
User 1 ■■■ * ActivityLog
```

Core distinction: the customer can reserve 2 Deluxe + 1 Executive without selecting physical room numbers. Reception later assigns actual rooms such as Deluxe → 101, Deluxe → 102, Executive → 201.

7. CORE BUSINESS RULES

BR-001: Anonymous booking is allowed without account creation.

BR-002: Registered customers use the same booking domain and can view their reservations in their dashboard.

BR-003: Registration may be encouraged but must not be mandatory for guest checkout.

BR-004: Anonymous booking lookup requires booking reference plus guest email.

BR-005: A booking can contain multiple room types and quantities; there is no one-room-per-booking assumption.

BR-006: Customers reserve room types/quantities; physical rooms are assigned separately.

BR-007: A room is unavailable for a stay when a conflicting active assignment/reservation occupies the requested date range.

BR-008: Check-in date is inclusive and check-out date is exclusive for night and overlap calculations.

BR-009: Check-out must be after check-in; nights are calculated server-side.

BR-010: Backend calculates prices and totals. Client-submitted totals are never authoritative.

BR-011: BookingRoomType stores the agreed rate snapshot.

BR-012: Booking status and payment status are separate.

BR-013: A booking may have multiple payment records; paid amount is calculated from valid payment records.

BR-014: Current MVP supports manual Cash, POS and Bank Transfer recording; online gateway integration is deferred.

BR-015: Check-in changes assigned rooms to Occupied; check-out releases them to Available unless another operational status applies.

BR-016: Check-in validates booking state, dates, assignments and configured payment policy.

BR-017: Check-out validates that the booking is currently checked in and follows the configured outstanding-balance policy.

BR-018: Cancelled bookings do not consume room availability.

BR-019: Customer, Receptionist and Administrator permissions are enforced by the backend.

BR-020: Important administrative and operational actions generate audit records.

BR-021: The initial 20 rooms are sample inventory only; administrators can add more.

BR-022: Booking/inventory changes use transactions and concurrency controls sufficient to prevent overbooking.

8. DTO / CONTRACT CATALOG

Area	Request DTOs	Response DTOs
Auth	RegisterRequest, LoginRequest	LoginResponse, CurrentUserResponse
Availability	AvailabilitySearchRequest	AvailabilityResponse, AvailableRoomTypeResponse
Booking	CreateBookingRequest, BookingLookupRequest, CancelBookingRequest	BookingResponse, BookingSummaryResponse, BookingDetailRes
Assignment	AssignRoomsRequest	RoomAssignmentResponse
Stay	CheckInRequest, CheckOutRequest	CheckInResponse, CheckOutResponse
Rooms	CreateRoomRequest, UpdateRoomRequest	RoomResponse, RoomListItemResponse
Room types	CreateRoomTypeRequest, UpdateRoomTypeRequest	RoomTypeResponse, RoomTypeListItemResponse
Payments	RecordPaymentRequest	PaymentResponse, PaymentListItemResponse
Profile	UpdateProfileRequest	ProfileResponse

Area	Request DTOs	Response DTOs
Notifications	MarkNotificationReadRequest or command route	NotificationResponse
Reports	ReportFilterRequest	DashboardResponse, ReportResponse

DTO rules

- Do not expose EF Core entities directly from controllers.
- Separate write DTOs from read DTOs where their shapes differ.
- Validate input at the API boundary, then enforce business rules in application/domain logic.
- Use lightweight list DTOs for tables and richer detail DTOs for detail pages.

9. API CONTRACT CATALOG

Area	Method	Route	Authorization
Auth	POST	/api/v1/auth/register	Public
Auth	POST	/api/v1/auth/login	Public
Auth	POST	/api/v1/auth/refresh	Refresh/auth
Auth	POST	/api/v1/auth/logout	Authenticated
Auth	GET	/api/v1/auth/me	Authenticated
Rooms	GET	/api/v1/room-types	Public
Rooms	GET	/api/v1/room-types/{id}	Public
Availability	POST	/api/v1/availability/search	Public
Booking	POST	/api/v1/bookings	Public/Auth
Booking lookup	POST	/api/v1/bookings/lookup	Public
Customer	GET	/api/v1/me/bookings	Customer
Customer	GET	/api/v1/me/bookings/{id}	Customer
Customer	POST	/api/v1/me/bookings/{id}/cancel	Customer
Customer	GET	/api/v1/me/profile	Customer
Customer	PUT	/api/v1/me/profile	Customer
Reception	GET	/api/v1/reception/bookings	Receptionist/Admin
Reception	GET	/api/v1/reception/bookings/{id}	Receptionist/Admin
Reception	POST	/api/v1/reception/bookings	Receptionist/Admin
Reception	POST	/api/v1/reception/bookings/{id}/assign-rooms	Receptionist/Admin
Reception	POST	/api/v1/reception/bookings/{id}/check-in	Receptionist/Admin
Reception	POST	/api/v1/reception/bookings/{id}/check-out	Receptionist/Admin
Reception	GET	/api/v1/reception/rooms	Receptionist/Admin
Reception	GET	/api/v1/reception/guests	Receptionist/Admin
Reception	POST	/api/v1/reception/payments	Receptionist/Admin
Admin	GET	/api/v1/admin/dashboard	Administrator
Admin	GET/POST	/api/v1/admin/rooms	Administrator
Admin	GET/PUT	/api/v1/admin/rooms/{id}	Administrator
Admin	PATCH	/api/v1/admin/rooms/{id}/status	Administrator

Area	Method	Route	Authorization
Admin	GET/POST	/api/v1/admin/room-types	Administrator
Admin	GET/PUT	/api/v1/admin/room-types/{id}	Administrator
Admin	PATCH	/api/v1/admin/room-types/{id}/status	Administrator
Admin	GET	/api/v1/admin/bookings	Administrator
Admin	GET	/api/v1/admin/guests	Administrator
Admin	GET/POST	/api/v1/admin/staff	Administrator
Admin	GET/PUT	/api/v1/admin/staff/{id}	Administrator
Admin	PATCH	/api/v1/admin/staff/{id}/status	Administrator
Admin	GET	/api/v1/admin/payments	Administrator
Admin	GET	/api/v1/admin/reports	Administrator
Admin	GET	/api/v1/admin/activity-logs	Administrator
Admin	GET	/api/v1/admin/notifications	Administrator
Admin	GET/PUT	/api/v1/admin/settings	Administrator

10. KEY CONTRACT EXAMPLES

AvailabilitySearchRequest

```
{ "checkInDate": "2026-10-10", "checkOutDate": "2026-10-15", "adults": 4, "children": 0, "rooms": 3 }
```

CreateBookingRequest

```
{ "checkInDate": "2026-10-10", "checkOutDate": "2026-10-15", "adults": 4, "children": 0, "guest": {
"firstName": "Grace", "lastName": "Ibrahim", "email": "grace@example.com", "phoneNumber":
"+2340000000000" }, "rooms": [ { "roomTypeId": "DELUXE-ID", "quantity": 2 }, { "roomTypeId":
"EXECUTIVE-ID", "quantity": 1 } ], "specialRequests": null }
```

AssignRoomsRequest

```
{ "assignments": [ { "bookingRoomTypeId": "LINE-1", "roomId": "ROOM-101" }, { "bookingRoomTypeId":
"LINE-1", "roomId": "ROOM-102" }, { "bookingRoomTypeId": "LINE-2", "roomId": "ROOM-201" } ] }
```

RecordPaymentRequest

```
{ "bookingId": "BOOKING-ID", "amount": 100000, "paymentMethod": "POS", "reference": "POS-REF-001",
"notes": "Initial deposit" }
```

11. STATE TRANSITIONS

Booking

```
Pending → Confirmed → CheckedIn → CheckedOut
Pending → Cancelled
Confirmed → Cancelled
```

The backend must reject illegal ordinary transitions such as CheckedOut → Pending.

Room

```
Available ↔ Occupied
Available → Maintenance → Available
Operational state → Inactive when retired.
```

Room availability for future dates is reservation/assignment driven and is not the same as current RoomStatus.

12. NON-FUNCTIONAL REQUIREMENTS

- Versioned ASP.NET Core API under /api/v1.
- PostgreSQL + Entity Framework Core.
- Secure identity and password handling.
- Server-side role authorization.
- Global exception handling with consistent error responses.
- Structured logging and validation.
- Transactions for multi-step inventory-changing operations.
- Concurrency protection against overbooking.
- Audit logging for important administrative/operational actions.
- Pagination, filtering and sorting for operational list endpoints where appropriate.
- No secrets or payment credentials committed to source control.

13. EXPLICITLY DEFERRED

- Paystack, Flutterwave or Stripe integration until the hotel provides an account/provider requirements.

- Payment gateway webhooks and automated reconciliation.
- Loyalty/reward points and membership tiers unless confirmed as actual requirements.
- Housekeeping module unless confirmed.
- Airport transfer and ancillary services unless confirmed.
- Dynamic/seasonal/weekend pricing, taxes, discounts and refunds until the hotel's policy is supplied.

14. OPEN BUSINESS DECISIONS

Decision	Current recommendation
Exact room catalogue	Admin-configurable; prototype room types are not authoritative.
Meaning of ■30k/■40k/■55k	Confirm whether these are room types, categories or pricing levels.
Taxes/fees	Do not add until hotel policy is confirmed.
Cancellation policy	Define cancellation window and refund rules.
No-show	Add NoShow status if required by hotel operations.
Check-in payment rule	Configurable; do not hardcode full payment without confirmation.
Check-out outstanding balance	Configurable; confirm hotel policy.
Room assignment	Assign before check-in is recommended.
Pending booking expiry	Define whether unpaid/pending reservations expire automatically.
Guest data retention	Define retention/privacy policy with the hotel.

15. SOURCE-OF-TRUTH GOVERNANCE

- This document is the backend design baseline.
- Frontend prototype data is not automatically real hotel data.
- Business-rule changes should be recorded here before implementation changes.
- Material domain/API changes require a new document version.
- Implementation should not invent new domain entities merely because a UI prototype contains decorative features.

Implementation readiness: The core model, enum set, base-class direction, DTO families, API route catalog, and principal business rules are defined. The remaining hotel-policy decisions should be confirmed before freezing this as the immutable production specification.